Sécurisation des communications

Prérequis :

- Généralités sur le web : adresse URL, adresse IP, serveur DNS, requête HTTP, modèle client/serveur
- Boucle for, fonctions, listes et dictionnaires sous Python
- les fonctions ord(), chr() de Python
- les portes logiques dont la porte XOR, les codages Unicode et ASCII

Objectifs :

- connaître le principe de chiffrement symétrique (clef partagée) et quelques exemples
- connaître le principe de chiffrement asymétrique (avec clef privée/clef publique) et comprendre le système RSA simplifié
- décrire l'échange d'une clef symétrique en utilisant un protocole asymétrique pour sécuriser une communication HTTPS

Si l'on utilise un site web pour effectuer des transactions bancaires, pour renseigner des informations personnelles (impôts, arrêt maladie, etc.), ou simplement pour exprimer son opinion, on souhaite que le contenu des messages envoyés ne soit connu que de deux entités : la source et la destination.

Pour qu'un message entre deux machines dans un réseau informatique ne puisse pas être compréhensible s'il est intercepté, il faut qu'il soit donc chiffré.

Il y a deux manières principales pour le faire :

- Par un **chiffrement symétrique** qui utilise une clé unique, connue seulement de l'émetteur et du récepteur. L'émetteur chiffre le message avec la clé et le récepteur le déchiffre avec la même clé.
- Par un **chiffrement asymétrique** qui utilise un couple de clés, l'une publique connue de tout le monde, l'autre privée connue uniquement par le récepteur. L'émetteur chiffre le message avec la clé publique. Le récepteur le déchiffre avec la clé privée.

Une vidéo de 9 minutes environ qui explique les principaux concepts de cette séance :

https://www.youtube.com/watch?v=7W7WPMX7arI&list=RDCMUC5cs06DgLFeyLIF_II7lWCQ&index=1

Point vocabulaire

Coder : représenter de l'information par un ensemble de signes prédéfinis. On utilise parfois le verbe *encoder.*

Décoder : interpréter un ensemble de signes pour extraire l'information qu'ils représentent.

Chiffrer : rendre une suite de symboles incompréhensible au moyen d'une *clef de chiffrement.*

Déchiffrer/décrypter : retrouver la suite de symboles originale à partir du message chiffré. On utilise le terme *déchiffrer* lorsque l'on utilise une *clef de déchiffrement* pour récupérer le texte initial et le terme *décrypter* lorsque l'on arrive à déterminer le message original sans utiliser la clef.

« Coder » et « décoder » s'utilisent lorsqu'il n'y a pas de secret : par exemple, « *codage en complément à deux* » des entiers. Le terme « coder » (qui est un anglicisme de « to code ») est aussi utilisé de façon informelle comme synonyme de « programmer » comme dans la phrase « j'ai codé cette application ». L'anglicisme « crypter » est à proscrire ; on utilisera « chiffrer ».

PARTIE I : Chiffrement symétrique

Dans un chiffrement symétrique, la clé utilisée par l'expéditeur pour chiffrer le message est la même que celle utilisée par le récepteur pour déchiffrer le message.

Un premier exemple : Le codage de César avec une clé de 13.

En ne considérant que les lettres en majuscule et non accentuées, notre alphabet se réduit à 26 lettres (ABCDEFGHIJKLMNOPQRSTUVWXYZ)
Avec un décalage de 13 : (A → N) et (N → A). On utilise bien la même clé, il s'agit donc d'un chiffrement symétrique.
Par exemple le message 'BONJOUR, COMMENT ALLEZ-VOUS' se code par :
'OBAWBHE, PBZZRAG NYYRM-IBHF' (seules les lettres ont été codées…) et le déchiffrement nous redonne bien le message d'origine.

Exercice 1 :

Compléter le programme ***C44_ex01.py*** *pour* réaliser ce chiffrement de César avec une clé de 13. On utilise le dictionnaire *d* ci-dessous qui permet de chiffrer et de déchiffrer un message :

d = {chr(i+65): chr((i+13)%26+65) for i in range(26)} # seules les lettres capitales sont chiffrées

● On rappelle que la fonction ***chr(i)*** renvoie la chaîne représentant un caractère dont le code de caractère Unicode est le nombre entier *i*. Par exemple, *chr(97)* renvoie la chaîne de caractères *'a'*, tandis que *chr(8364)* renvoie *'€'*.
L'intervalle valide pour cet argument est de 0 à 1 114 111 (0x10FFFF en base 16). Une exception ValueError est levée si *i* est en dehors de l'intervalle.

● On pourra utiliser la méthode *get* : ***dict.get(keyname,value)*** : renvoie la valeur de la clé *keyname* ou la valeur *value* si la clé spécifiée n'est pas dans le dictionnaire.

Un second exemple : Utilisation de XOR

XOR est une opération de logique bit à bit qui renvoie 0 si les deux bits sont égaux et 1 sinon.

Par exemple : XOR(13,6)=11 car 13=$(1101)_2$, 6=$(0110)_2$ et XOR(1101_2,0110_2) = $(1011)_2$ = 11

Une particularité de cette opération est : Si a XOR *b* = *c* alors a XOR c = *b* et *b* XOR c = *a*

Ce qui permet un chiffrement symétrique.

On vérifie avec l'exemple précédent que : XOR(13,11) = XOR(1101_2,1011_2) = $(0110)_2$ = 6

XOR(6,11) = XOR(0110_2,1011_2) = $(1101)_2$ = 13

Prenons le mot 'bonjour' dont le code Unicode est donné par ce programme :

```
m = "bonjour"
for c in m:
    print(ord(c), end=' ')
```

La fonction *ord()* renvoie la valeur représentant l'Unicode d'un caractère spécifié : 98 111 110 106 111 117 114

Le mot 'nsi' va nous servir de clé : ('nsi' → 110 115 105)

La méthode consiste alors à aligner le mot et la clé en la répétant autant de fois que nécessaire et d'effectuer un XOR entre les codes Unicode en binaire. On pratique donc un XOR lettre par lettre entre les nombres obtenus en binaire et on traduit en décimale l'Unicode binaire obtenu.

Compléter le tableau correspondant ci-dessous :

Message	**b**	**o**	**n**	**j**	**o**	**u**	**r**
Unicode en décimale	98	111	110	106	111	117	114
Unicode en binaire	0110 0010						
Clé	**n**	**s**	**i**	**n**	**s**	**i**	**n**
Unicode en décimale	110	115	105	110	115	105	110
Unicode en binaire	0110 1110						
Message chiffré	**XOR(b,n)**	**XOR(o,s)**	**XOR(n,i)**	**XOR(j,n)**	**XOR(o,s)**	**XOR(u,i)**	**XOR(r,n)**
Unicode en binaire	0000 1100						
Unicode en décimale	12						

Pour décoder le message, il suffit alors de recommencer l'opération avec les codes Unicode du message chiffré en utilisant la même clé : (12 xor 110 → 98 donc 'b') etc.

En Python l'opérateur ∧ permet d'effectuer un XOR directement sur deux entiers. Le programme suivant chiffre le message précédent en utilisant cette méthode :

```
message = "bonjour"
cle = "nsi"

def chiffre(message,key):
    c = []
    n = len(message)
    m = len(key)
    j = 0
    for i in range(n):
        c.append(ord(message[i]) ^ ord(key[j]))
        j = (j+1)%m
    return c

print(chiffre(message,cle))
```

Exercice 2 :

1. Ouvrir le programme ***C44_ex02.py*** et tester la fonction *chiffre* (message et clé différents de l'exemple précédent). Noter les six premiers nombres obtenus. Codent-ils tous des caractères imprimables ?
   Si non, dire de façon générale ce qu'ils codent ?

   Aide :
https://fr.wikipedia.org/wiki/American_Standard_Code_for_Information_Interchange#Groupement_par_type_de_caract%C3%A8res

2. Compléter dans le programme la fonction *dechiffre* qui prend en paramètres le message chiffré et la clé et qui renvoie le message déchiffré. Tester ensuite cette fonction pour retrouver le message initial.

Avantages et inconvénients :

L'algorithme de chiffrage donné ci-dessus possède les avantages suivants :

- L'opération "ou exclusif" se calcule très rapidement.
- Le chiffrement fonctionne avec n'importe quel fichier binaire et pas seulement du texte : par exemple avec une image.
- La même clef sert au chiffrement et au déchiffrement.

Par contre ce genre de chiffrement possède un énorme inconvénient : il nécessite que la clef soit partagée entre l'émetteur et le récepteur tout en restant secrète. Ce qui pose des problèmes de logistique : la clef doit être transmise préalablement, par un autre canal.

Dans le cas de transmission sur Internet, le chiffrement est bien sûr beaucoup plus sophistiqué, car le chiffrement XOR n'est pas d'une grande sécurité. Des algorithmes de chiffrement comme AES ou ChaCha20 sont considérés comme sûrs et sont efficaces, mais ils supposent que les participants puissent échanger la clé symétrique de façon sûre, ce que nous allons étudier dans la 2e partie avec les algorithmes de chiffrement asymétrique.

PARTIE II : Chiffrement asymétrique

Exemple du code de César

Il suffit de choisir une autre clé que 13 pour obtenir un chiffrement asymétrique, c'est-à-dire où la clé de chiffrement et la clé de déchiffrement ne sont pas identiques.
Si la clé de chiffrement est 9, la clé de déchiffrement est de 26 − 9 = 17.

Exercice 3 :

Écrire un programme qui chiffre et déchiffre le message 'HELLO LES QUICHES' pour illustrer le chiffrement asymétrique avec le code de César. Enregistrer ce programme sous *C44_ex03.py*.

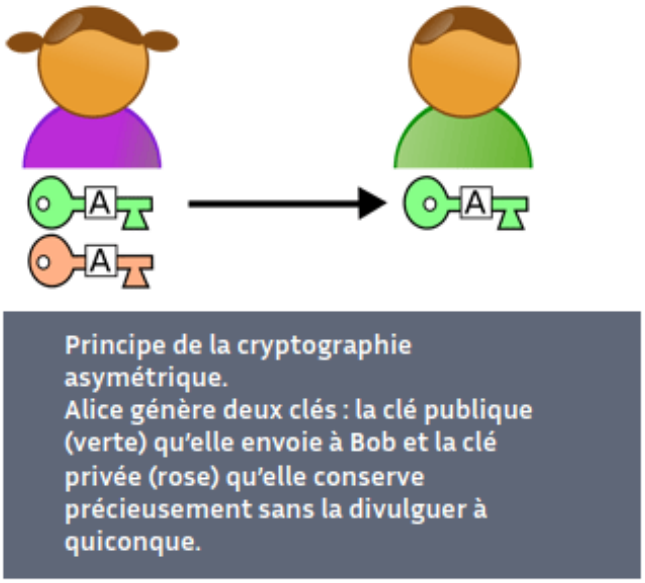
Le système RSA simplifié

Dans la pratique, les systèmes utilisés sont beaucoup plus élaborés que celui énoncé précédemment. Un système de chiffrement très courant, le système RSA, repose sur la difficulté de factoriser un nombre en produit de deux nombres premiers. Son principe utilise des théorèmes d'arithmétique dont le détail va au-delà du programme de terminale.

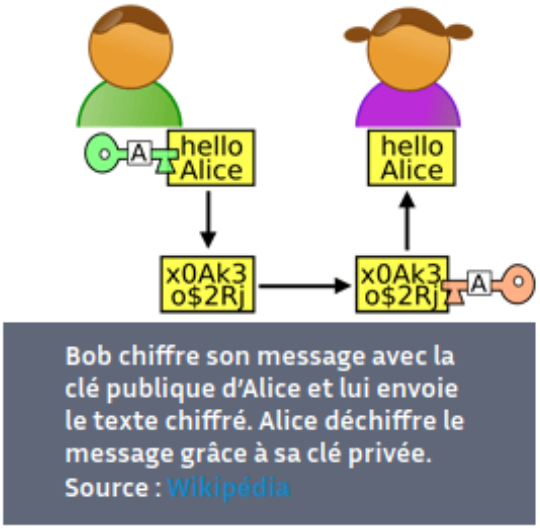
Il a été inventé en 1977 par trois mathématiciens Ron **R**ivest, Adir **S**hamir et Len **A**dleman, qui se sont inspirés des idées sur les échanges de clés contenus dans un article des cryptologues Whitfield Diffie et Martin Hellman paru en 1976.

Le système RSA est aujourd'hui un système universel servant dans une multitude d'applications. Sa technique est protégée par un brevet dans certains pays (aux États-Unis, ce brevet a expiré en septembre 2000). Elle a été vendue à près de 350 entreprises et on estime que plus de 300 millions de programmes installés peuvent utiliser le RSA, les transactions sécurisées via internet par exemple l'emploient pour la plupart. Internet fait un usage systématique du RSA pour assurer la confidentialité du courrier électronique et authentifier les utilisateurs.

Le système RSA est un **système de cryptographie asymétrique (ou à clé publique)** basé sur l'existence d'une fonction dite à sens unique, pour transformer un message en message codé. Il faut que cette fonction soit simple à appliquer à un quelconque message, mais qu'il soit difficile de retrouver le message original à partir du message codé. La cryptographie à clé publique permet de coder un message secret et aussi d'authentifier l'émetteur d'un message.



Principe de la cryptographie asymétrique.
Alice génère deux clés : la clé publique (verte) qu'elle envoie à Bob et la clé privée (rose) qu'elle conserve précieusement sans la divulguer à quiconque.



Bob chiffre son message avec la clé publique d'Alice et lui envoie le texte chiffré. Alice déchiffre le message grâce à sa clé privée.
Source : Wikipédia

La clé publique peut être connue de tous et donc mise sur une page web, envoyée par mail, etc.

Méthode

Étape 1 : préparation des clés

● *Choisir deux nombres p et q premiers et grands ($p, q > 10^{100}$)*

● *Calculer* $n = p \times q$ et $f = (p-1) \times (q-1)$.

● *Choisir un nombre e premier avec f et strictement inférieur à f.*

Le couple (e, n) constitue la clé publique *connue de tout le monde qui permet de chiffrer un message m.*

● *Déterminer un entier d tel que le reste de la division euclidienne de* $e \times d$ *par f soit 1. Des méthodes mathématiques permettent de déterminer d.* ***L'entier d est la clé privée*** *uniquement connue par le récepteur.*

Étape 2 : chiffrement du message

● *Soit un message m à chiffrer connu sous la forme d'un entier naturel (par exemple en utilisant le code ASCII des caractères).*

● *Si m < n alors le message chiffré x est le reste de la division euclidienne de* m^e *par n. Sinon, le message est découpé en sous-messages codés par des nombres m tels que m < n. Chaque entier m doit être premier avec n.*

Étape 3 : déchiffrement du message

● *calculer le reste de la division euclidienne de* x^d *par n. Ce reste est le message initial m sous la forme d'un entier.*

Exemple facile

Étape 1 : préparation des clés

Soit deux nombres premier $p = 7$ et $q = 17$: $n = p \times q = 7 \times 17 = 119$ et $f = (p-1) \times (q-1) = 6 \times 16 = 96$
On choisit un nombre premier avec f et strictement inférieur à f : $e = 5$
La clé publique pour notre exemple est : $(e,n) = (5, 119)$
On obtient $d = 77$ qui est la clé privée (on verra une méthode algorithmique dans l'exercice 4)

Étape 2 : chiffrement du message

Pour chiffrer la lettre ‘A’ dont le code ASCII est 65, on détermine le reste de la division Euclidienne de 65e par *n* soit le reste de la division euclidienne de 65^5 par 119. Dans le shell de Pyzo : (65**5) % 119 et on obtient 46. La lettre ‘A’ est donc chiffrée par le nombre $x = 46$

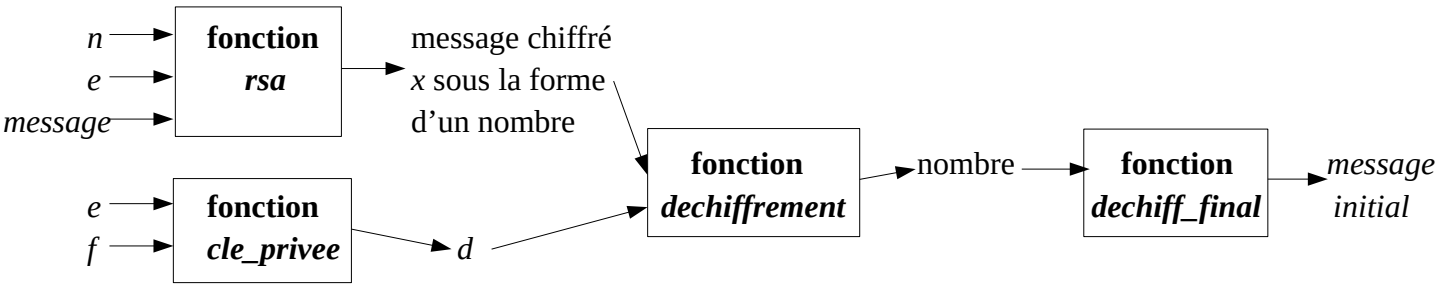
Étape 3 : déchiffrement du message

Pour déchiffrer le message $x = 46$, on calcule le reste de la division euclidienne de 46^d par *n* soit le reste de la division euclidienne de 46^{77} par 119. Dans le shell de Pyzo : (46**77) % 119 et on obtient 65. On retrouve bien le code ASCII de la lettre ‘B’.

Remarque : la sécurité du chiffrement réside dans la difficulté à trouver *p* et *q* à partir de *n* quand *n* comprend plusieurs centaines de chiffres. Sans les entiers *p* et *q*, on ne peut pas calculer *f* et on ne peut pas déterminer la clé privée *d*.

Exercice 4 :

L’objectif est de compléter un programme qui code le système RSA simplifié en Python.
Ce programme comprend quatre fonctions illustrées par le schéma ci-dessous :



1. On donne $p = 6\,991$ et $q = 7\,529$. Calculer *n* et *f*.
   On choisit $e = 23$. Vérifier que ce nombre est premier avec *f*.

2. Compléter dans le programme ***C44_ex04*** la fonction ***rsa*** qui prend en paramètres une chaîne de caractères *message (le message initial à chiffrer)* ainsi que les nombres *e* et *n* et renvoie le message chiffré qui est *(m**e) % n*.
   Par exemple, si *message* = ‘RSA’, à l’aide de la fonction *ord*, chaque lettre est remplacée par son code ASCII et on obtient les entiers 82, 83 et 65 qui permettent de constituer le nombre 828365.
   Chiffrer alors le message ‘RSA’.

3. Compléter la fonction ***cle_privee*** qui prend en paramètres *e* et *f* et renvoie le nombre *d*.

Comme l'algorithme du PGCD étendu n'est pas au programme, on va procéder ainsi pour la recherche de l'entier d :
- on initialise d à 1
- tant que d est strictement inférieur à f : si le reste de la division euclidienne de $e \times d$ par f est égal à 1, on retourne d, sinon on incrémente d de 1.

4. Lire le code de la fonction ***dechiffrement*** qui prend en paramètres un message chiffré x, les nombres d et n, et renvoie un nombre, par exemple 828365 si le message est 'RSA'.
Expliquer en détail ce que fait cette fonction et pourquoi elle ne calcule pas directement x^d.

5. Compléter la fonction ***dechiff_final*** qui prend en paramètre un nombre tel que 828365 et renvoie le texte correspondant, soit 'RSA' dans ce cas. On suppose que les codes ASCII sont constitués de deux chiffres.

PARTIE III : Le protocole HTTPS

Le système RSA permet donc de chiffrer des communications en se protégeant contre l'espionnage des communications, c'est-à-dire l'observation passive par un tiers des messages envoyés entre deux participants. Mais il reste deux problèmes à régler : l'attaque de l'homme du milieu et l'impossibilité d'utiliser le système RSA pour chiffrer directement des gros volumes de données et des flux de communication comme une visioconférence.

Attaque de l'homme du milieu

Supposons qu'Alice soit cliente de l'établissement bancaire BobBanque. Les communications entre Alice et BobBanque étant confidentielles, elles doivent être évidemment chiffrées mais cela n'est pas suffisant.

Marion (la malveillante) peut procéder à l'attaque suivante. Elle envoie un e-mail à Alice, lui demandant de se connecter d'urgence sur le site de la banque. Si Alice n'est pas méfiante et que Marion utilise en plus d'autres techniques comme de reproduire exactement la page d'accueil du site de la banque, alors elle ne se rendra pas compte qu'elle est connectée au mauvais site. Marion pourra recevoir tous les messages envoyés par Alice, les retransmettre passivement ou en les modifiant (par exemple changer le montant et le destinataire d'un virement bancaire !) au véritable site de la banque.

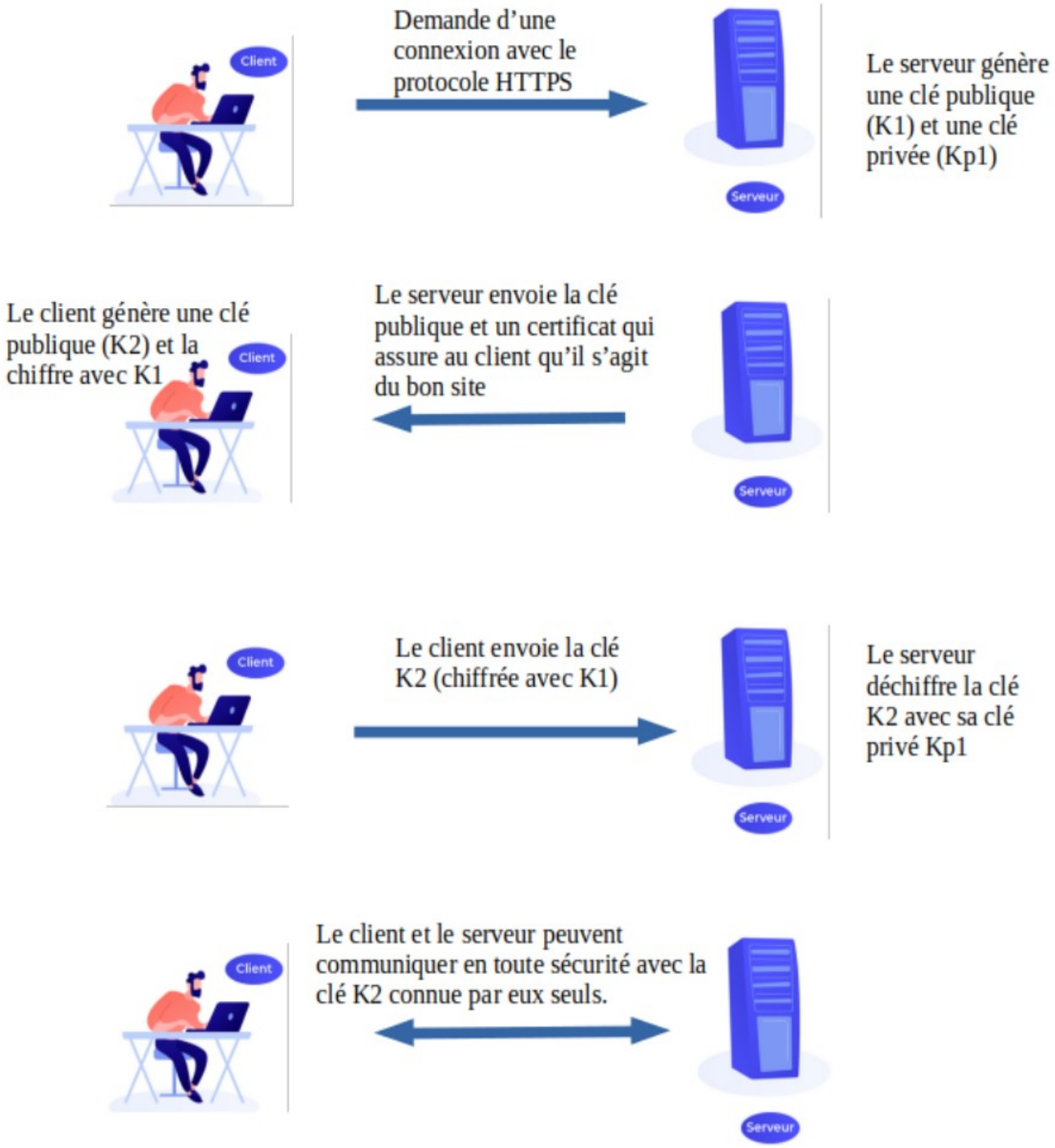
Ici, le chiffrement n'est d'aucune aide. On appelle cela une **attaque de l'homme du milieu** (ou **man in the middle attack** en anglais : MITM). La faille fondamentale est l'absence d'authentification. Le serveur de Marion, qui imite le comportement du serveur de BobBanque n'a pas eu à prouver qu'il était bien une machine appartenant à BobBanque. Le protocole HTTPS permet à chaque participant de s'assurer de l'identité de son correspondant en utilisant le système RSA comme système d'authentification.

Des calculs trop coûteux

Le chiffrement et le déchiffrement du système RSA nécessitent des calculs très coûteux ce qui rend impossible de chiffrer directement des gros volumes de données et des flux de communication comme une visioconférence.

La solution est de choisir une clé pour un algorithme de chiffrement symétrique qui est un fichier de quelques milliers d'octets. Elle peut être envoyée de manière sûre en utilisant RSA. Une fois la clé symétrique reçue, un algorithme de chiffrement symétrique efficace peut être utilisé. Les deux méthodes de chiffrement (symétrique et asymétrique) sont donc utilisées lorsqu'un navigateur demande une page web avec le protocole HTTPS.

Schéma des échanges de communications entre un client et un serveur avec le protocole HTTPS.



Lors du protocole **HTTPS** (**H**yper**T**ext **Tr**ansfer **P**rotocole **S**ecure, protocole de transfert de documents hypertexte sécurisé), les échanges entre client et serveur se font donc avec un chiffrement symétrique (clé K2) après que celle-ci ait été échangée avec un chiffrement asymétrique.

Ce protocole est compatible avec HTTP : le but n'est pas de recréer un nouveau protocole, mais de rendre HTTP plus sûr. Il est donc simplement la réunion de deux protocoles existants, **HTTP** (pour l'interaction avec le serveur) et **TLS** (Transport Layer Security, sécurité de la couche de transport) qui ajoute une phase permettant l'authentification du serveur et la mise en place sécurisée d'une clé de chiffrement symétrique, appelée clé de session.

Il utilise des certificats envoyés par le serveur web et signés par des **autorités de certification (AC)** : ce sont des entités habilitées à délivrer des certificats numériques :

- des entreprises spécialisées ;
- des associations à but non lucratif (comme *Let's Encrypt*, libre et gratuite)
- des états.

Elles sont en nombre relativement restreint. Les systèmes d'exploitation et les navigateurs web, en particulier, possèdent les clés publiques des AC sous forme de fichiers. Par exemple, au début 2020, la fondation Mozilla, qui produit le navigateur web libre Firefox, reconnaît environ un peu plus d'une centaine d'autorités de certification. Chacune d'elles répond à des critères très stricts, vérifiés par des audits réguliers dont les résultats sont publics.

Exercice 5 :

1. Consulter la page d'accueil du site de la société informatique de France.
   L'adresse est : https://www.societe-informatique-de-france.fr/
   Préciser les informations disponibles sur le certificat de sécurité, quels sont les chiffrements utilisés, symétrique et asymétrique.

2. Même question en téléchargeant le fichier https://www.nsi-terminale.fr/cert.pem
   cert.pem est un fichier de type certificat X.509 (3,5 Ko).

Exercice 6 :

On considère le cas d'un site Internet https://www.monsite.fr. Ce dernier a été mal configuré et le fichier privkey.pem, contenant la clé privée correspondant à la clé publique contenue dans le certificat, est téléchargeable. Le site a choisi d'utiliser RSA pour l'échange de la clé de session (clé symétrique).

Indiquer ce que peut faire un utilisateur malveillant en possession de cette clé privée.